

面向对象简介



Python完全采用了面向对象的思想，是真正面向对象的编程语言，完全支持面向对象的基本功能，例如：继承、多态、封装等。

Python中，一切皆对象。我们在前面学习的数据类型、函数等，都是对象。

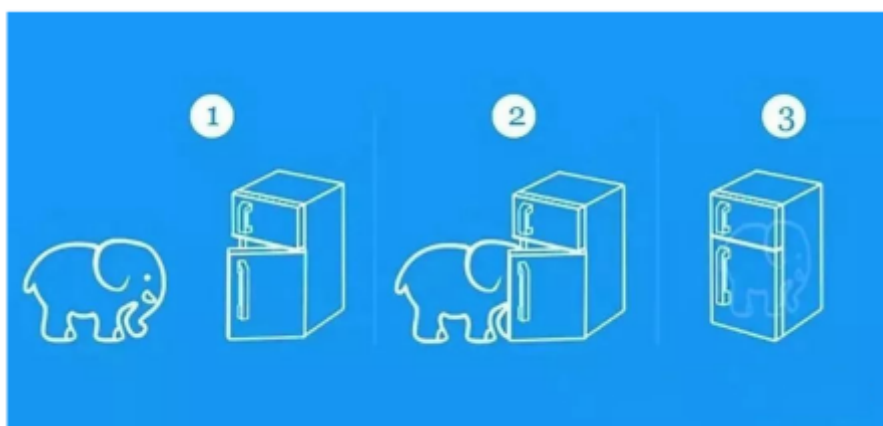


- 面向对象（Object oriented Programming, OOP）编程的思想主要是针对大型软件设计而来的。
- 面向对象编程使程序的扩展性更强、可读性更好，使编程可以像搭积木一样简单。

- 面向对象编程将数据和操作数据相关的方法封装到对象中，组织代码和数据的方式更加接近人的思维，从而大大提高了编程的效率。

♡ Python支持面向过程、面向对象、函数式编程等多种编程范式。

面向过程和面向对象思想



面向过程和面向对象的区别

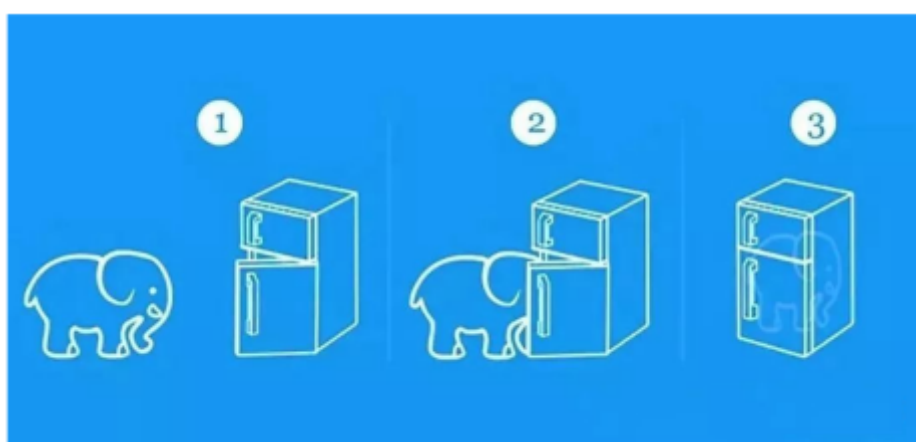
面向过程和面向对象都是对软件分析、设计和开发的一种思想,它指导着人们以不同的方式去分析、设计和开发软件。C语言是一种典型的面向过程语言，Java是一种典型的面向对象语言。

面向过程是什么？

面向过程适合简单、不需要协作的事务，重点关注如何执行。面向过程时，我们首先思考“怎么按步骤实现？”。比如，如何开车？我们很容易就列出实现步骤：



比如：把大象装冰箱分几步？



但是当我们思考比较复杂的设计任务时，比如“如何造车？”，就会发现列出1234这样的步骤，是不可能的。那是因为，造车太复杂，需要很多协作才能完成。此时面向对象思想就应运而生了。

面向对象是什么？



面向对象(Oriented-Object)思想更契合人的思维模式。我们首先思考的是“怎么设计这个事物？”。比如思考造车，我们就会先思考“车怎么设计？”，而不是“怎么按步骤造车的问题”。这就是思维方式的转变。

天然的，我们就会从“车由什么组成”开始思考：



为了协作，我们找轮胎厂完成制造轮胎的步骤，发动机厂完成制造发动机的步骤；这样，发现大家可以同时进行车的制造，最终进行组装，大大提高了效率。

具体到轮胎厂的一个流水线操作，仍然是有步骤的，还是离不开执行者、离不开面向过程！

面向对象可以帮助我们从宏观上把握、从整体上分析整个系统。但是，具体到实现部分的微观操作（就是一个个方法），仍然需要面向过程的思路去处理。

我们千万不要把面向过程和面向对象对立起来。他们是相辅相成的。面向对象离不开面向过程！

面向对象和面向过程总结

- ① 都是解决问题的思维方式，都是代码组织的方式。
- ② 面向过程是一种“执行者思维”，解决简单问题可以使用面向过程
- ③ 面向对象是一种“设计者思维”，解决复杂、需要协作的问题可以使用面向对象

面向对象离不开面向过程：

- 宏观上：通过面向对象进行整体设计
- 微观上：执行和处理数据，仍然是面向过程

实时效果反馈

1. 如下关于python中，面向对象的说法，错误的是：

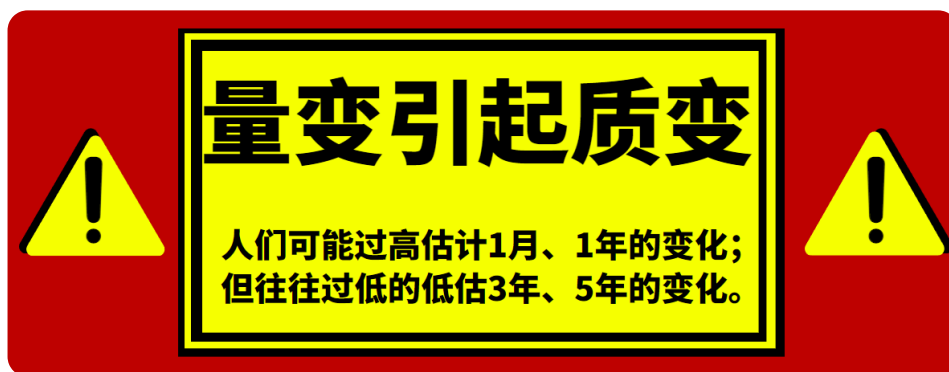
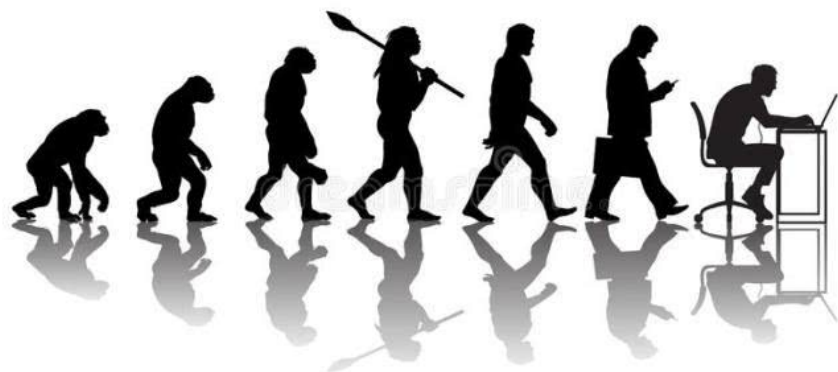
- A** python中，一切都是对象
- B** python中，不是所有的都是对象。比如：数字就不是对象。
- C** 面向过程是一种“执行者思维”，解决简单问题可以使用面向过程
- D** 面向对象是一种“设计者思维”，解决复杂、需要协作的问题可以使用面向对象

答案

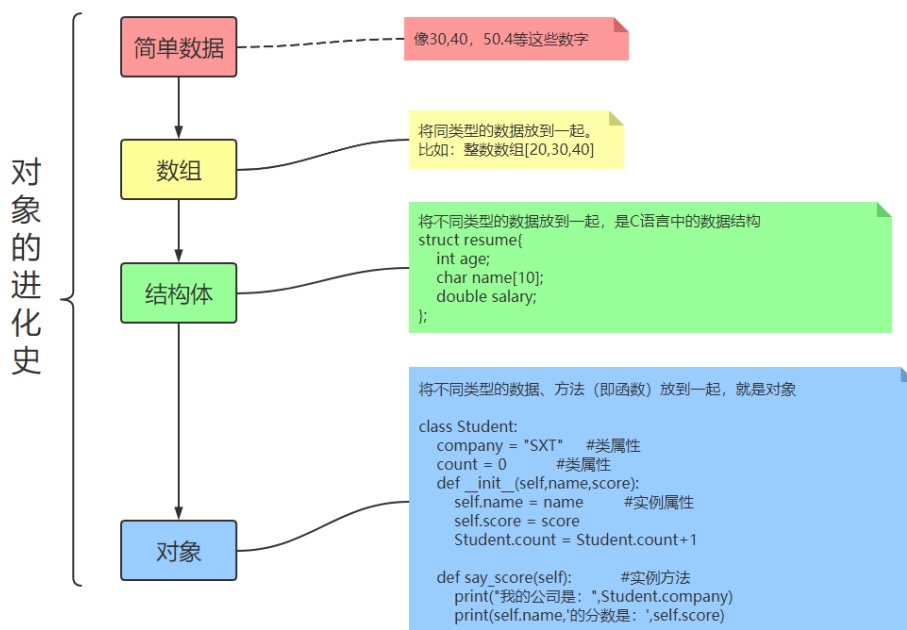
1=>B

对象的进化

人类进化：环境变化，适者生存



随着编程面临的问题越来越复杂，编程语言本身也在进化，从主要处理简单数据开始，随着数据变多进化“数组”；数据类型变复杂，进化出了“结构体”；处理数据的方式和逻辑变复杂，进化出了“对象”。



1 简单数据

像30,40, 50.4等这些数字，可以看做是简单数据。最初的计算机编程，都是像这样的数字。

2 C语言中的数组

将同类型的数据放到一起。比如：整数数组 [20,30,40]，浮点数数组 [10.2, 11.3, 12.4]，字符串数组： ["aa", " bb", " cc"]

⚠ 上面的[20,30,40]不是python中的列表，是C语言中的数组

3 C语言中的结构体

将不同类型的数据放到一起，是C语言中的数据结构。比如：

```

1 struct resume{
2     int age;
3     char name[10];
4     double salary;
5 };
  
```

4 对象

将不同类型的数据、方法（即函数）放到一起，就是对象。比如：

```

1 class Student:
2     company = "SXT"          #类属性
3     count = 0                #类属性
4     def __init__(self, name, score):
5         self.name = name      #实例属性
6         self.score = score
7         Student.count = Student.count+1
8
9     def say_score(self):      #实例方
10        法
11        print("我的公司是: ", Student.company)
12        print(self.name, '的分数是: ', self.score)

```

类的定义



类可以看做是一个模版，或者图纸，系统根据类的定义来造出对象。我们要造一个汽车，怎么样造？类就是这个图纸，规定了汽车的详细信息，然后根据图纸将汽车造出来。

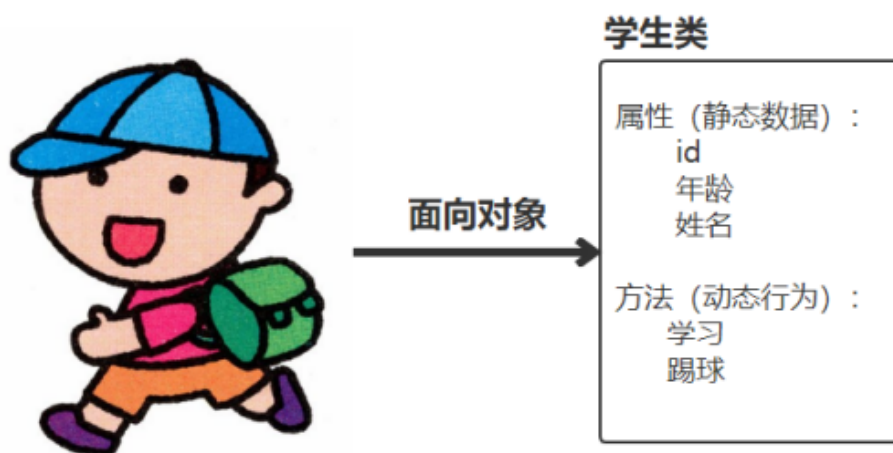
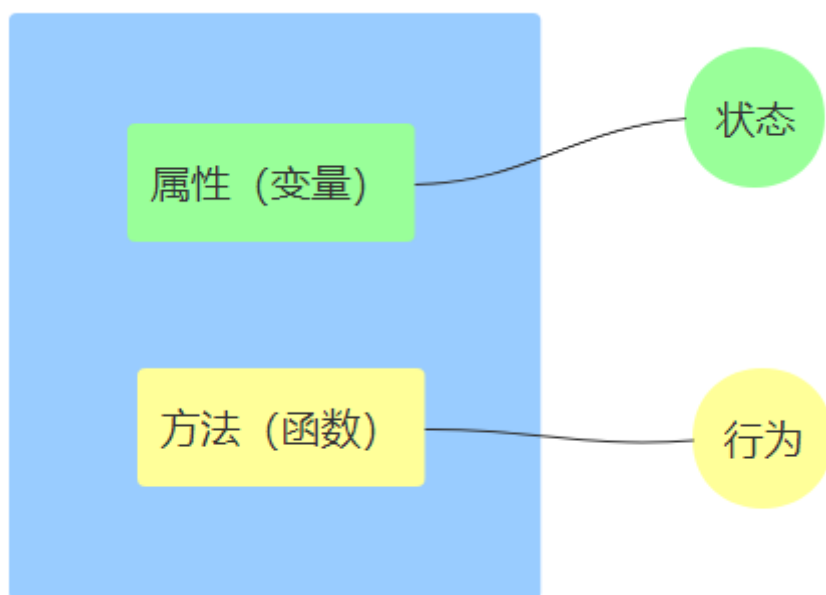
类：我们叫做 `class`。对象：我们叫做 `object`，`instance` (实例)。以后我们说某个类的对象，某个类的实例。是一样的意思。

我们把对象比作一个“饼干”，类就是制造这个饼干的“模具”。



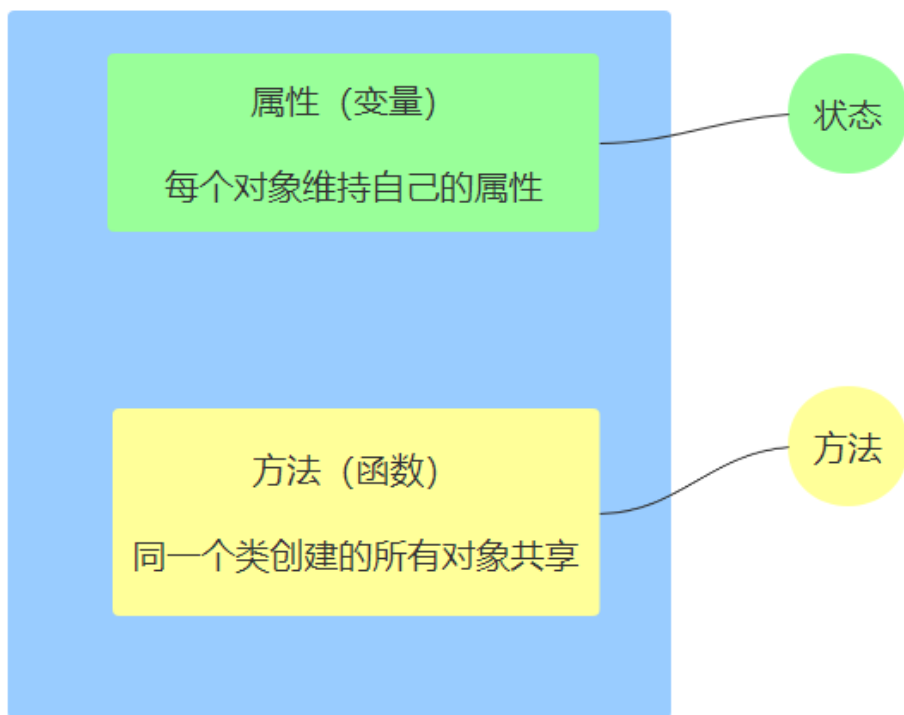
属性和方法

我们通过类定义数据类型的属性（数据）和方法（行为），也就是说，“类将行为和状态打包在一起”。



对象是类的具体实体，一般称为“类的实例”。类看做“饼干模具”，对象就是根据这个“模具”制造出的“饼干”。

从一个类创建对象时，每个对象会共享这个类的行为（类中定义的方法），但会有自己的属性值（不共享状态）。更具体一点：“方法代码是共享的，属性数据不共享”。



♥ Python中，“一切皆对象”。类也称为“类对象”，类的实例也称为“实例对象”。

定义类的语法格式如下：

```
1 class 类名:  
2     类体
```

要点如下：

- ① 类名必须符合“标识符”的规则；一般规定，首字母大写，多个单词使用“驼峰原则”。
- ② 类体中我们可以定义属性和方法
- ③ 属性用来描述数据，方法(即函数)用来描述这些数据相关的操作

【操作】一个典型的类的定义

```
1 class Student:
2     def __init__(self, name, score): #构造方法第
    一个参数必须为self
3         self.name = name           #实例属性
4         self.score = score
5
6     def say_score(self):             #实例方法
7         print("{0}的分数是
    {1}".format(self.name, self.score))
8
9 s1 = Student('张三', 80) #s1是实例对象，自动调用
    __init__()方法
10 s1.say_score()
```

`pass` 为空语句。就是表示什么都不做，只是作为一个占位符存在。当你写代码时，遇到暂时不知道往方法或者类中加入什么时，可以先用`pass`占位，后期再补上。

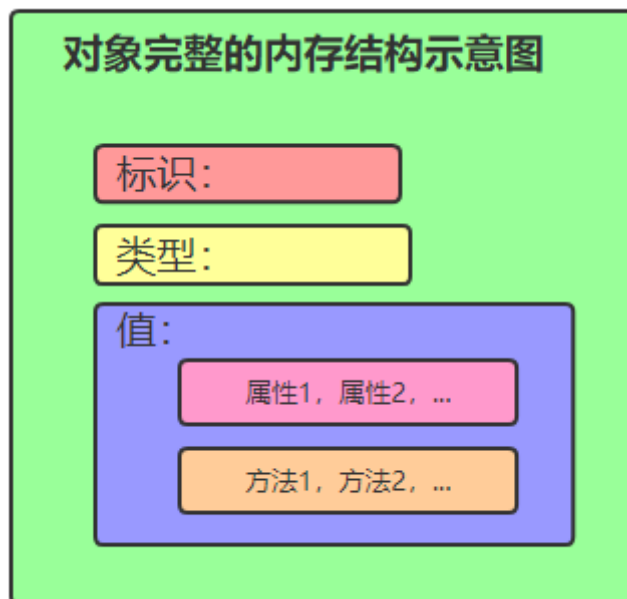


对象完整内存结构

类是抽象的，也称之为“对象的模板”。我们需要通过类这个模板，创建类的实例对象，然后才能使用类定义的功能。

我们前面说过一个Python对象包含三个部分：`id`（identity识别码）、`type`（对象类型）、`value`（对象的值）。

现在，我们可以更进一步的说，一个Python对象包含如下部分：



实时效果反馈

1. 如下是一个典型类的定义代码和调用代码，说法错误的是：

```

1 class Student:
2     def __init__(self, name, score):
3         self.name = name
4         self.score = score
5
6     def say_score(self):
7         print("{0}的分数是
8         {1}".format(self.name, self.score))
9
10 s1 = Student('张三', 80)
11 s1.say_score()

```

- A `__init__` 构造方法第一个参数可以不是self
- B `name` 和 `score` 是属性
- C `say_score(self)` 是方法
- D `s1 = Student('张三', 80)` , 其中s1是实例对象, 自动调用 `__init__()` 方法

答案

1=>A

`__init__` 构造方法和 `__new__` 方法

初始化对象，我们需要定义构造函数 `__init__()` 方法。构造方法用于执行“实例对象的初始化工作”，即对象创建后，初始化当前对象的相关属性，无返回值。



构造方法是负责初始化（装修），不是建对象（房子）

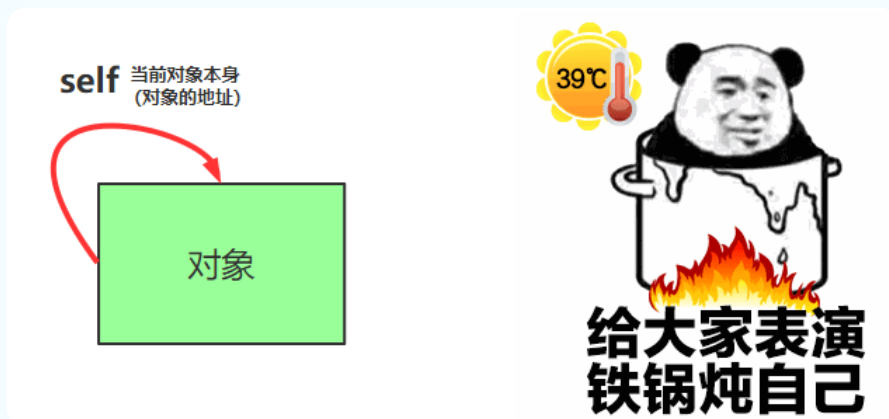
`__init__` 的要点如下：

- ① 名称固定，必须为：`__init__`
- ② 第一个参数固定，必须为：`self`。`self` 指的就是刚刚创建好的实例对象
- ③ 构造函数通常用来初始化实例对象的实例属性，如下代码就是初始化实例属性：`name` 和 `score`

```
1 def __init__(self, name, score):
2     self.name = name           #实例属性
3     self.score = score
```

- ④ 通过“类名(参数列表)”来调用构造函数。调用后，将创建好的对象返回给相应的变量。比如：`s1 = Student('张三', 80)`
- ⑤ `__init__` 方法：初始化创建好的对象，初始化指的是：“给实例属性赋值”
- ⑥ `__new__` 方法：用于创建对象，但我们一般无需重定义该方法
- ⑦ 如果我们不定义 `__init__` 方法，系统会提供一个默认的 `__init__` 方法。如果我们定义了带参的 `__init__` 方法，系统不创建默认的 `__init__` 方法

♥ Python中的 `self` 相当于C++中的 `self` 指针，JAVA和C#中的 `this` 关键字。Python中，`self` 必须为构造函数的第一个参数，名字可以任意修改。但一般惯例，都叫做 `self`



实时效果反馈

1. 如下关于 `__init__` 构造方法和 `__new__` 方法，说法错误的是：

- A 通过“类名(参数列表)”来调用构造函数 `__init__()`
- B `__init__()` 方法：用于创建对象
- C `__new__()` 方法：用于创建对象，但我们一般无需重定义该方法
- D `__init__()` 方法：第一个参数固定，必须为：`self`。`self` 指的就是刚刚创建好的实例对象

答案

1=>B

实例属性和实例方法

实例属性

实例属性是从属于实例对象的属性，也称为“实例变量”。他的使用有如下几个要点：

- 1 实例属性一般在 `__init__()` 方法中通过如下代码定义：

```
self.实例属性名 = 初始值
```

- 2 在本类的其他实例方法中，也是通过 `self` 进行访问：

```
self.实例属性名
```

- 3 创建实例对象后，通过实例对象访问：

```
obj01 = 类名()
```

#创建和初始化对象，调用 `__init__()` 初始化属性

```
obj01.实例属性名 = 值
```

#可以给已有属性赋值，也可以新加属性

```
1 class Student:
2     def __init__(self, name, score):
3         self.name = name        #增加name属性
4         self.score = score      #增加score属性
5
6     def say_score(self):
7         self.age = 18           #增加age属性
8         print("{0}的分数是
9         {1}".format(self.name, self.score))
10
11 s1 = Student("张三", 80)
12 s1.say_score()
13 print(s1.age)
14 s1.salary = 3000               #s1对象增加salary属性
15
16 s2 = Student("李四", 90)
17 s2.say_score()
18 print(s2.age)
```

实时效果反馈

1. 如下代码，说法错误的是：

```
1 class Student:
2     def __init__(self, name, score):
3         self.name = name
4         self.score = score
5
6     def say_score(self):
7         print(self.name, '的分数
8         是: ', self.score)
9
10 s1 = Student('张三', 80)
11 s1.say_score()
```

- A** `name` 和 `score` 是实例属性
- B** `say_score(self)` 是实例方法
- C** `say_score(self)` 中可以调用实例属性 `name` 和 `score`
- D** `say_score(self)` 中不可以调用实例属性 `name` 和 `score`

答案

1=>D

实例方法

实例方法是从属于实例对象的方法。实例方法的定义格式如下：

```

1 def 方法名(self [, 形参列表]):
2     函数体

```

方法的调用格式如下：

对象.方法名([实参列表])

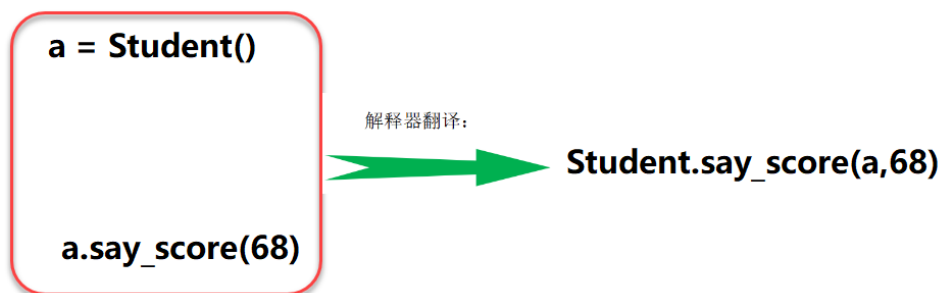
要点：

- 1 定义实例方法时，第一个参数必须为 `self`。和前面一样，`self` 指当前的实例对象。
- 2 调用实例方法时，不需要也不能给 `self` 传参。`self` 由解释器自动传参

函数和方法的区别

- 1 都是用来完成一个功能的语句块，本质一样。
- 2 方法调用时，通过对象来调用。方法从属于特定实例对象，普通函数没有这个特点
- 3 直观上看，方法定义时需要传递 `self`，函数不需要

实例对象的方法调用本质



其他操作

- 1 `dir(obj)` 可以获得对象的所有属性、方法
- 2 `obj.__dict__` 对象的属性字典
- 3 `pass` 空语句
- 4 `isinstance (对象,类型)` 判断“对象”是不是“指定类型”

实时效果反馈

1. 如下代码，说法错误的是：

```
1 class Student:
2     def say_hello(self):
3         print(self, '----', "hello")
4
5 s1 = Student()
6 s1.say_hello()
7 Student.say_hello(s1)
```

- A `say_hello(self)` 是实例方法，第一个参数必须是 `self`
- B `say_hello(self)` 是实例方法，第一个参数可以不是 `self`
- C `Student.say_hello(s1)` 的作用和 `s1.say_hello()` 本质是一样的
- D 可以正常执行完毕

答案

1=>B

类对象、类属性、类方法、静态方法

类对象

我们在前面讲的类定义格式中，`class 类名:`。实际上，当解释器执行 `class` 语句时，就会创建一个类对象

【操作】测试类对象的生成

```
1 class Student:
2     pass      #空语句
3
4 print(type(Student))
5 print(id(Student))
6
7 Stu2 = Student
8 s1 = Stu2()
9 print(s1)
```

执行结果如下：

```
1 <class 'type'>
2 51686328
3 <__main__.Student object at
  0x0000000002B5FDD8>
```

我们可以看到实际上生成了一个变量名就是类名 `Student` 的对象。我们通过赋值给新变量 `Stu2`，也能实现相关的调用。说明，确实创建了“类对象”。

实时效果反馈

1. 执行如下代码，说法错误的是：

```
1 class Student:
2     pass      #空语句
3
4 Stu2 = Student
5 s1 = Stu2()
```

A 只是定义了类 `Student`，不会生成任何对象

- B** 执行 `class` 语句时，就会创建一个类对象
- C** `pass` 为空语句
- D** `Stu2` 和 `Student` 变量都指向了同一个类对象

答案

1=>A

类属性

类属性是从属于“类对象”的属性，也称为“类变量”。由于，类属性从属于类对象，可以被所有实例对象共享。

类属性的定义方式：

```
1 class 类名:
2     类变量名 = 初始值
```

在类中或者类的外面，我们可以通过：`类名.类变量名` 来读写

内存分析实例对象和类对象创建过程（重要）

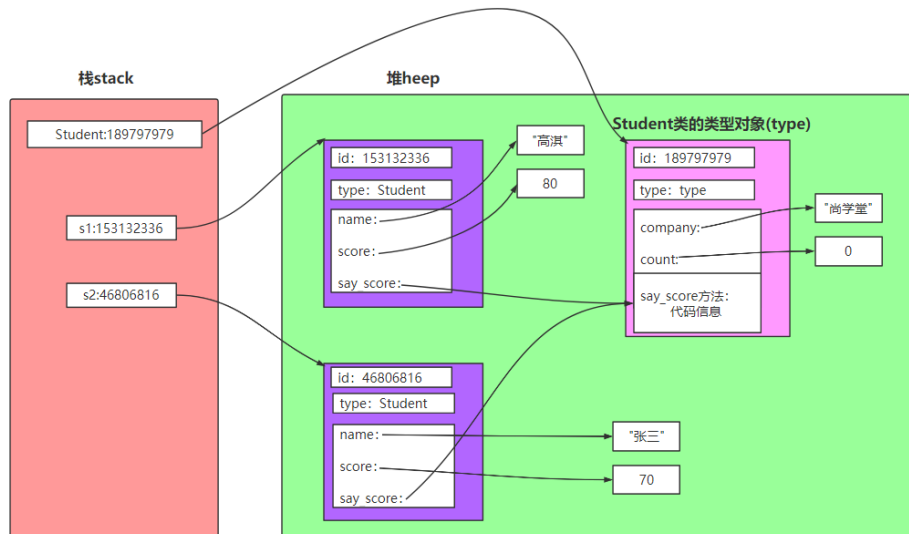
我们以下面代码为例，分析整个创建过程，让大家对面向对象概念掌握更加深刻：

```
1 class Student:
2     company = "尚学堂" # 类属性
3     count = 0 # 类属性
4
5     def __init__(self, name, score):
6         self.name = name # 实例属性
```

```

7         self.score = score
8         Student.count = Student.count + 1
9
10        def say_score(self): # 实例方法
11            print("我的公司是: ", Student.company)
12            print(self.name, '的分数是: ',
13                  self.score)
14
15        s1 = Student('高淇', 80) # s1是实例对象, 自动调用__init__()方法
16        s2 = Student('张三', 70)
17        s1.say_score()
18        print('一共创建{0}个Student对象'.format(Student.count))

```



实时效果反馈

1. 如下代码，说法错误的是：

```

1 class Student:
2     company = "SXT"      #类属性
3     count = 0           #类属性
4
5     def __init__(self, name, score):
6         self.name = name    #实例属性
7         self.score = score

```

- A `company` 是类属性
- B `count` 是类属性
- C `name` 是实例属性
- D `score` 是类属性，也是实例属性

答案

1=>D

类方法

类方法是从属于“类对象”的方法。类方法通过装饰器 `@classmethod` 来定义，格式如下：

```

1 @classmethod
2 def 类方法名(cls [, 形参列表]) :
3     方法体

```

要点如下：

- 1 `@classmethod` 必须位于方法上面一行
- 2 第一个 `cls` 必须有；`cls` 指的就是“类对象”本身
- 3 调用类方法格式：`类名.类方法名(参数列表)`。参数列表中，不需要也不能给 `cls` 传值
- 4 类方法中访问实例属性和实例方法会导致错误

5 子类继承父类方法时，传入 `cls` 是子类对象，而非父类对象(△ 讲完继承再说)

【操作】类方法使用测试

```
1 class Student:
2     company = "SXT"      #类属性
3
4     @classmethod
5     def printCompany(cls):
6         print(cls.company)
7
8 Student.printCompany()
```

静态方法

Python中允许定义与“类对象”无关的方法，称为“静态方法”。

“静态方法”和在模块中定义普通函数没有区别，只不过“静态方法”放到了“类的名字空间里面”，需要通过“类调用”。



静态方法通过装饰器@staticmethod来定义，格式如下：

```

1 @staticmethod
2 def 静态方法名([形参列表]) :
3     方法体

```

要点如下：

- ① `@staticmethod` 必须位于方法上面一行
- ② 调用静态方法格式：`类名.静态方法名(参数列表)`
- ③ 静态方法中访问实例属性和实例方法会导致错误

【操作】静态方法使用测试

```

1 class Student:
2     company = "SXT" # 类属性
3
4     @staticmethod
5     def add(a, b): # 静态方法
6         print("{0}+{1}={2}".format(a,b,
7         (a+b)))
8         return a+b
9 Student.add(20,30)

```

实时效果反馈

1. 如下代码是测试类方法和静态方法，说法错误的是：

```

1 class Student:
2     company = "SXT"      #类属性
3
4     @classmethod
5     def printCompany(cls):
6         print(cls.company)
7
8     @staticmethod
9     def add(a, b):      # 静态方法
10        print("{0}+{1}={2}".format(a,b,
11        (a+b)))
12        return a+b

```

- A** 通过类名可以直接调用类方法: `Student.printCompany()`
- B** 通过类名可以直接调用静态方法: `Student.add(20,30)`
- C** “静态方法”和在模块中定义普通函数没有区别, 只不过“静态方法”放到了“类的名字空间里面”, 需要通过“类调用”
- D** 定义类方法, `cls` 可以没有; `cls` 指的就是“类对象”本身

答案

1=>D

`__del__`方法(析构函数)和垃圾回收机制



`__del__()` 称为“析构方法”，用于实现对象被销毁时所需的操作。比如：释放对象占用的资源，例如：打开的文件资源、网络连接等。

Python实现自动的垃圾回收，当对象没有被引用时（引用计数为0），由垃圾回收器调用 `__del__()`。

我们也可以通过 `del` 语句删除对象，从而保证调用 `__del__()`。

系统会自动提供 `__del__` 方法，一般不需要自定义析构方法。

```

1  #析构函数
2  class Person:
3
4      def __del__(self):
5          print("销毁对象: {0}".format(self))
6
7  p1 = Person()
8  p2 = Person()
9  del p2
10 print("程序结束")

```

运算结果：

```

1 销毁对象: <__main__.Person object at
    0x02175610>
2 程序结束
3 销毁对象: <__main__.Person object at
    0x021755D0>

```

实时效果反馈

1. 关于python中 `__del__()` 析构函数，说法错误的是：

- A** `__del__()` 称为“析构方法”，用于实现对象被销毁时所需的操作
- B** Python自动垃圾回收，当对象没有被引用时，由垃圾回收器调用 `__del__()`
- C** 通过 `del` 语句删除对象，不会调用 `__del__()`
- D** 系统会自动提供 `__del__` 方法，一般不需要自定义析构方法

答案

1=>C

`__call__` 方法和可调用对象

- ① Python 中，凡是可以将 `()` 直接应用到自身并执行，都称为可调用对象。
- ② 可调用对象包括自定义的函数、Python 内置函数、以及本节所讲的实例对象。
- ③ 定义了 `__call__()` 的对象，称为“可调用对象”，即该对象可以像函数一样被调用。
- ④ 该方法使得实例对象可以像调用普通函数那样，以“对象名`()`”的形式使用。

```

1  def f1():
2      print("f1")
3
4  f1()    #本质也是调用了__call__()方法
5
6  class Car:
7      def __call__(self, age, money):
8          print("__call__方法")
9          print("车龄: {0}, 金额:
10         {1}".format(age, money))
11
12 f2 = Car()
13 f2(3, 200000)    #像调用函数那样调用, 本质也是调用
                    了__call__()

```

运行结果:

```

1  f1
2  车龄: 3, 金额: 200000

```

实时效果反馈

1. 关于 `__call__` 方法和可调用对象, 说法错误的是:

- ☒ A 可以将 () 直接应用到自身并执行, 都称为可调用对象
- ☐ B 自定义函数就是可调用对象, 本质就是调用了 `__call__` 方法
- ☐ C `__call__` 方法, 使得实例对象可以像调用普通函数那样, 以“对象名()”的形式使用

D `__call__` 方法，使得实例对象可以像调用普通函数那样，以“类名()”的形式使用

答案

1=>D

方法没有重载



如果我们在类体中定义了多个重名的方法，只有最后一个方法有效。

建议：不要使用重名的方法！Python中方法没有重载。

- 在其他一些语言(比如：Java)中，可以定义多个重名的方法，只要保证方法签名唯一即可。方法签名包含3个部分：方法名、参数数量、参数类型。
- Python中，方法的参数没有声明类型（调用时确定参数的类型），参数的数量也可以由可变参数控制。因此，Python中是没有方法的重载的。

- 1 **#Python**中没有方法的重载。定义多个同名方法，只有最后一个有效
- 2 **class Person:**

```

3     def say_hi(self):
4         print("hello")
5
6
7     def say_hi(self, name):
8         print("{0},hello".format(name))
9
10 p1 = Person()
11
12 #p1.say_hi()          #不带参，报错：TypeError:
    say_hi() missing 1 required positional
    argument: 'name'
13
14 p1.say_hi("高淇")

```

方法的动态性



Python是动态语言，我们可以动态的为类添加新的方法，或者动态的修改类的已有的方法

```

1 #测试方法的动态性
2 class Person:
3     def work(self):
4         print("努力上班！")
5
6     def play_game(self):

```

```
7     print("玩游戏")
8
9     def work2(s):
10         print("好好工作，努力上班！")
11
12
13     Person.play = play_game
14     Person.work = work2
15
16     p = Person()
17     p.play()
18     p.work()
```

我们可以看到，`Person` 动态的新增了 `play_game` 方法，以及用 `work2` 替换了 `work` 方法

实时效果反馈

1. 关于python的方法，说法错误的是：

- ☐ A Python中方法没有重载
- ☐ B 可以动态的为类添加新的方法
- ☐ C 可以动态的修改类的已有的方法
- ☐ D Python中方法也可以重载

答案

1=>D

私有属性和私有方法(实现封装)



Python对于类的成员没有严格的访问控制限制，这与其他面向对象语言有区别。关于私有属性和私有方法，有如下要点：

- ① 通常我们约定，两个下划线开头的属性是私有的(private)。其他为公共的(public)。
- ② 类内部可以访问私有属性(方法)
- ③ 类外部不能直接访问私有属性(方法)
- ④ 类外部可以通过 `类名_私有属性(方法)名` 访问私有属性(方法)

⚠️【注】方法本质上也是属性！只不过是可以通过()执行而已。

所以，此处讲的私有属性和公有属性，也同时讲解了私有方法和公有方法的使用。

如下测试中，同时也包含了私有方法和公有方法的例子。

【测试】私有属性和公有属性使用测试

```

1  #测试私有属性、私有方法
2  class Employee:
3      __company = "百战程序员" #私有.通过dir查到
   __Employee__company
4
5      def __init__(self, name, age):
```

```

6         self.name = name
7         self.__age = age                #私有实例属性
8
9         def say_company(self):
10             print("我的公司
是: ", Employee.__company)            #类内部可以
直接访问私有属性
11             print(self.name, "的年龄
是: ", self.__age)
12             self.__work()
13
14         def __work(self): #私有实例方法，通过dir可查
到__Employee__work
15             print("工作！好好工作，好好赚钱，娶个媳
妇！")
16
17 p1 = Employee("高淇", 32)
18 print(p1.name)
19 print(dir(p1))                        #
20 p1.say_company()
21 print(p1.__Employee__age)            #通过这种方式可以
直接访问到私有属性 。通过dir可以查到属性：
__Employee__age
22 #print(p1.__age)                    #直接访问私有属性，报
错
23 #p1.__sleep()                      #直接访问私有方法，报
错

```

执行结果：

```

1 高淇
2 ['_Person__age', '_Person__leg_num',
   '_Person__sleep', '__class__', '__delattr__',
   '__dict__', '__dir__', '__doc__', '__eq__',
   '__format__', '__ge__', '__getattribute__',
   '__gt__', '__hash__', '__init__',
   '__init_subclass__', '__le__', '__lt__',
   '__module__', '__ne__', '__new__',
   '__reduce__', '__reduce_ex__', '__repr__',
   '__setattr__', '__sizeof__', '__str__',
   '__subclasshook__', '__weakref__', 'name',
   'say_age']
3 腿的数目: 2
4 高淇 的年龄是: 18
5 睡觉
6 18

```

从打印的 `Person` 对象所有属性我们可以看出。私有属性 `_age` 在实际存储时是按照 `_Person__age` 这个属性来存储的。这也就是为什么我们不能直接使用 `_age` 而可以使用 `_Person__age` 的根本原因。

实时效果反馈

1. 关于python中的私有属性和私有方法，说法错误的是：

```

1 class Employee:
2     __company = "百战程序员" #私有.通过dir查到
   __Employee__company
3
4     def __work(self): #私有实例方法，通过dir可查
   到__Employee__work
5         print("工作！好好工作，好好赚钱，娶个媳
   妇！")

```

A 通常约定，两个下划线开头的属性是私有的(private)。其他为公共的(public)

B `__company` 是私有属性

C `__work(self)` 是私有方法

D python中，私有属性和私有方法很严格，外部绝不可能访问

答案

1=>D

@property 装饰器

`@property` 可以将一个方法的调用方式变成“属性调用”。

`@property` 主要用于帮助我们处理属性的读操作、写操作。对于某一个属性，我们可以直接通过：

```
emp1.salary = 30000
```

如上的操作读操作、写操作。但是，这种做法不安全。比如，我需要限制薪水必须为 1-10000 的数字。这时候，我们就需要通过使用装饰器 `@property` 来处理。

```

1  #测试@property
2  class Employee:
3
4      def __init__(self, name, salary):
5          self.name = name
6          self.__salary = salary
7
8      @property                                #相当于salary属性
      的getter方法
9      def salary(self):
10         print("月薪为{0}, 年薪为
      {1}".format(self.__salary,
      (12*self.__salary)))
11         return self.__salary;
12
13     @salary.setter
14     def salary(self, salary):                #相当于salary
      属性的setter方法
15         if(0<salary<1000000):
16             self.__salary = salary
17         else:
18             print("薪水录入错误! 只能在0-1000000
      之间")
19
20
21  emp1 = Employee("高淇", 100)
22  print(emp1.salary)
23
24  emp1.salary = -200

```

运行结果:

- 1 月薪为100,年薪为1200
- 2 100
- 3 月薪为100,年薪为1200
- 4 100
- 5 薪水录入错误! 只能在0-1000000之间

属性和方法命名总结

- 1 `_xxx` : 保护成员, 不能用 `from module import *` 导入, 只有类对象和子类对象能访问这些成员。
(⚠ 讲完模块再看)
- 2 `__xxx__` : 系统定义的特殊成员
- 3 `__xxx` : 类中的私有成员, 只有类对象自己能访问, 子类对象也不能访问。(但, 在类外部可以通过 `对象名._类名__xxx` 这种特殊方式访问。Python不存在严格意义的私有成员)

⚠ 再次强调, 方法和属性都遵循上面的规则。

类编码风格

- 1 类名首字母大写, 多个单词之间采用驼峰原则。
- 2 实例名、模块名采用小写, 多个单词之间采用下划线隔开
- 3 每个类, 应紧跟“文档字符串”, 说明这个类的作用
- 4 可以用空行组织代码, 但不能滥用。在类中, 使用一个空行隔开方法; 模块中, 使用两个空行隔开多个类

实时效果反馈

1. 关于属性和方法命名规则, 说法错误的是:

- A 类名首字母大写，多个单词之间采用驼峰原则
- B 实例名、模块名采用小写，多个单词之间采用下划线隔开
- C `__xxx__`：系统定义的特殊成员
- D `__xxx`：类中的私有成员，只有类对象自己能访问，子类对象也不能访问。Python中，存在严格意义的私有成员，外部永远无法访问。

答案

1=>D

关于None和判断的总结

None, Null, 空



None是什么？

- ① 与C和JAVA不同，python中是没有 `NULL` 的，取而代之的是 `None`。
- ② `None` 是一个特殊的常量，表示变量没有指向任何对象。
- ③ 在Python中，`None` 本身实际上也是对象，有自己的类型 `NoneType`。
- ④ 你可以将 `None` 赋值给任何变量，但我们不能创建 `NoneType` 类型的对象

```

1 obj = None
2 obj2 = None
3 print(type(None))
4 print(id(None))
5 print(id(obj))
6 print(id(obj2))

```

执行结果：

```

1 <class 'NoneType'>
2 140717958924280
3 140717958924280
4 140717958924280

```

⚠ None不是False，None不是0，None不是空字符串。None和任何其他的数据类型比较永远返回False。

None和其他类型的比较

- None和其他任何类型比较都会返回False

```

1 a = None
2 if a is None and a==None:
3     print("a是None")           #会执行
4 if a==False or a==0:
5     print("None不等于False")   #不会被打
    印

```

- 空列表、空字符串、0之间的比较

- ① if语句判断时,空列表[], 空字典{}, 空元组(), 0等一系列代表空和无的对象会被转换成False

```
1 a=[];b=();c={};d="";e=0;f=None
2 if (not a) and (not b) and (not c) and
   (not d) and (not e) and (not f):
3     print("if判断时, 空列表[], 空字符串、0、
       None等代表空和无的对象会被转换成False")
```

- ② == 和 is 判断时, 空列表、空字符串不会自动转成 False

```
1 a=[];b=();c={};d="";e=0;
2 if (a==False or d==False):
3     print("==时, 空列表、空字符串不是
       False! ")    #不会执行
4 if(e==False):
5     print("==时, 0会转成False")
```

实时效果反馈

1. Python中None的说法, 错误的是:

- A None是一个特殊的常量, 表示变量没有指向任何对象
- B None本身不是对象, 有自己的类型NoneType
- C 可以将None赋值给任何变量
- D None和其他任何类型比较都会返回False

答案

1=>B

实操作业

- ① 如下代码，使用图文分析整个内存过程：

```

1  class Student:
2      company = "尚学堂"          #类属性
3      count = 0                   #类属性
4
5      def __init__(self, name, score):
6          self.name = name        #实例属性
7          self.score = score
8          Student.count = Student.count+1
9
10     def say_score(self):         #实例方
11         法
12         print("我的公司是：", Student.company)
13         print(self.name, '的分数是：', self.score)
14
15 s1 = Student('高淇', 80)         #s1是实例对
16     象，自动调用__init__()方法
17 s1.say_score()
18 print('一共创建{0}个Student对
19 象'.format(Student.count))

```

- ② 设计一个名为MyRectangle的矩形类来表示矩形。这个类包含：
- (1) 左上角顶点的坐标：x, y

(2) 宽度和高度: width、height

(3) 构造方法: 传入x, y, width, height。如果 (x, y) 不传则默认是0, 如果width 和height不传, 则默认是100.

(4) 定义一个getArea() 计算面积的方法

(5) 定义一个getPerimeter(), 计算周长的方法

(6) 定义一个draw()方法, 使用海龟绘图绘制出这个矩形